Liberté • Égalité • Fraternité
RÉPUBLIQUE FRANÇAISE

MINISTÈRE CHARGÉ
DE L'EMPLOI

DOSSIER PROFESSIONNEL (DP)

Nom de naissance ▸ **MBALA**
Nom d'usage ▸ **MBALA**
Prénom ▸ **Hermann**
Adresse ▸ **Fleury-Mérogis (91)**

Titre professionnel visé

Développeur Web et Web Mobile

MODALITÉ D'ACCÈS :

☒ Parcours de formation
☐ Validation des Acquis de l'Expérience (VAE)

Présentation du dossier

Le dossier professionnel (DP) constitue un élément du système de validation du titre professionnel. **Ce titre est délivré par le Ministère chargé de l'emploi.**

Le DP appartient au candidat. Il le conserve, l'actualise durant son parcours et le présente **obligatoirement à chaque session d'examen**.

Pour rédiger le DP, le candidat peut être aidé par un formateur ou par un accompagnateur VAE.

Il est consulté par le jury au moment de la session d'examen.

Pour prendre sa décision, le jury dispose :

1. des résultats de la mise en situation professionnelle complétés, éventuellement, du questionnaire professionnel ou de l'entretien professionnel ou de l'entretien technique ou du questionnement à partir de productions.
2. du **Dossier Professionnel** (DP) dans lequel le candidat a consigné les preuves de sa pratique professionnelle
3. des résultats des évaluations passées en cours de formation lorsque le candidat évalué est issu d'un parcours de formation
4. de l'entretien final (dans le cadre de la session titre).

[Arrêté du 22 décembre 2015, relatif aux conditions de délivrance des titres professionnels du ministère chargé de l'Emploi]

Ce dossier comporte :

- pour chaque activité-type du titre visé, un à trois exemples de pratique professionnelle ;
- un tableau à renseigner si le candidat souhaite porter à la connaissance du jury la détention d'un titre, d'un diplôme, d'un certificat de qualification professionnelle (CQP) ou des attestations de formation ;
- une déclaration sur l'honneur à compléter et à signer ;
- des documents illustrant la pratique professionnelle du candidat (facultatif)
- des annexes, si nécessaire.

Pour compléter ce dossier, le candidat dispose d'un site web en accès libre sur le site.

http://travail-emploi.gouv.fr/titres-professionnels

Sommaire

Exemples de pratique professionnelle

1. Développer la partie front-end d'une application web ou web mobile sécurisée p. 5

- Installer et configurer son environnement de travail en fonction du projet web ou web mobile p. 5
- Maquetter des interfaces utilisateur web ou web mobile p. 9
- Réaliser des interfaces utilisateur statiques web ou web mobile p. 14
- Développer la partie dynamique des interfaces utilisateur web ou web mobile p. 18

2. Développer la partie back-end d'une application web ou web mobile sécurisée p. 21

- Mettre en place une base de données relationnelle p. 21
- Développer des composants d'accès aux données SQL et NoSQL p. 25
- Développer des composants métier côté serveur p. 28
- Documenter le déploiement d'une application dynamique web ou web mobile p. 35

Titres, diplômes, CQP, attestations de formation *(facultatif)* p. 38

Déclaration sur l'honneur p. 39

Documents illustrant la pratique professionnelle *(facultatif)* p. 40

Annexes *(Si le RC le prévoit)* p. 41

EXEMPLES DE PRATIQUE PROFESSIONNELLE

Activité-type 1

Développer la partie front-end d'une application web ou web mobile sécurisée

Exemple n°1 ▸ *Installer et configurer son environnement de travail en fonction du projet web ou web mobile*

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

Je travaille sur un Chromebook. Pour développer, j'ai activé l'environnement Linux du Chromebook (Debian 12) et j'y ai installé tous mes outils en ligne de commande.

Installation des outils :

- **Visual Studio Code**, mon éditeur de code, que j'ouvre sur un projet avec la commande `code ~/innovshop`.
- **PHP 8.2** et ses extensions, puis **Composer**, le gestionnaire de dépendances PHP.
- **MariaDB**, le serveur de base de données. J'ai créé un utilisateur dédié à chaque projet, avec des droits uniquement sur sa propre base.
- **Node.js et npm**, pour les projets JavaScript.
- **Git**, pour suivre les versions de mon code, et un compte **GitHub** pour les sauvegarder en ligne.
- **Symfony CLI**, l'outil en ligne de commande de Symfony.

Vérification et création du projet InnovShop :

1. J'ai vérifié chaque outil avec `php -v`, `composer --version`, `git --version` et `symfony version`.
2. J'ai lancé `symfony check:requirements`, qui confirme que mon ordinateur a tout ce qu'il faut pour Symfony.
3. J'ai choisi **Symfony 7.4 LTS** : la version 8 demande PHP 8.4, alors que j'ai PHP 8.2. La version LTS est en plus maintenue longtemps.
4. J'ai créé le projet avec `symfony new innovshop --version="7.4.*" --webapp`, puis lancé le serveur local avec `symfony server:start`.

Configuration et sécurité :

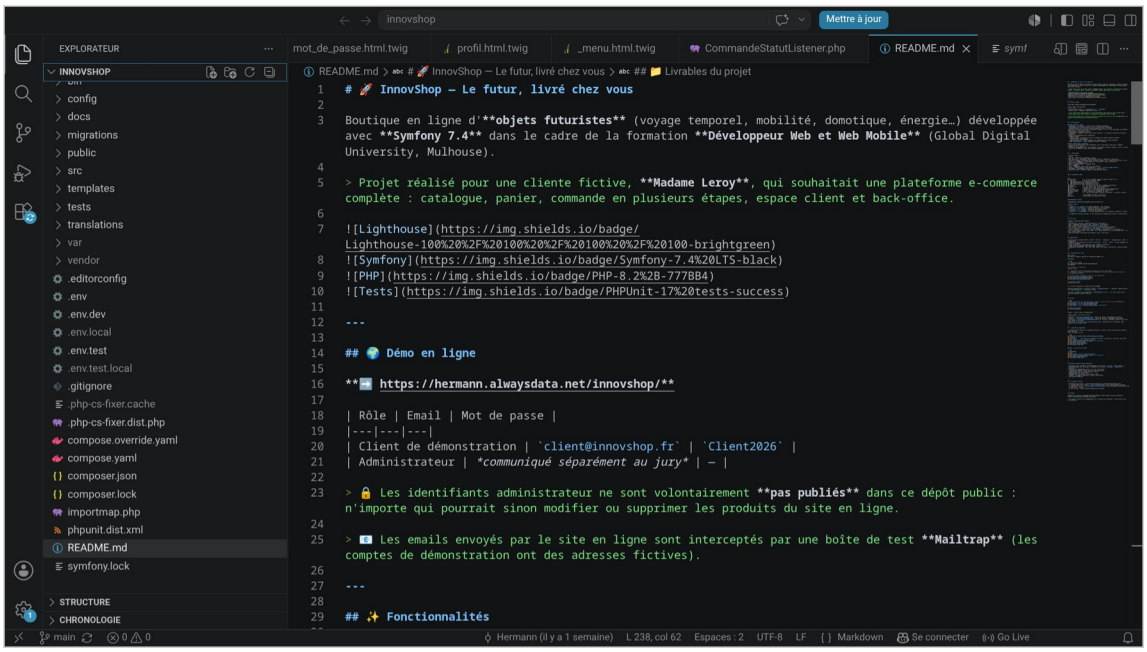
- J'ai mis la connexion à la base de données dans un fichier `.env.local`, qui n'est jamais envoyé sur GitHub grâce au `.gitignore`. Mes mots de passe restent donc privés.
- J'ai créé le dépôt GitHub et pris l'habitude de faire un commit avec un message clair à la fin de chaque étape, puis un `git push`.

Problèmes rencontrés et résolus :

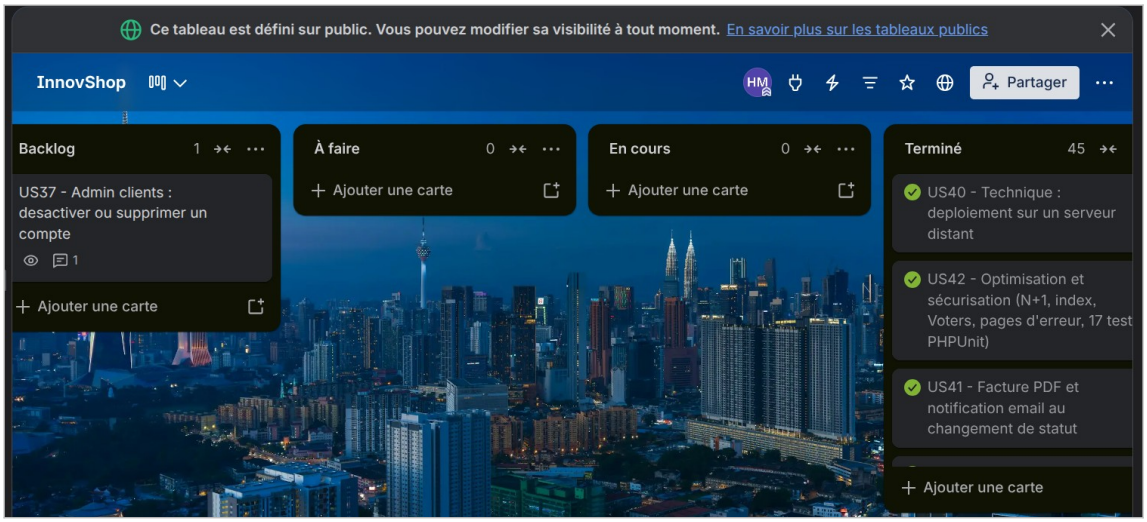
- L'environnement Linux refusait de démarrer (erreur 62, mise à jour nécessaire) : un redémarrage complet du Chromebook a résolu le problème.
- Le processeur de mon Chromebook ne gère pas certaines instructions (AVX), ce qui empêche d'installer certains outils en local. J'ai appris à vérifier la compatibilité d'un outil avant de l'installer, et à garder les outils

qui marchent déjà plutôt que d'en ajouter d'autres, comme XAMPP.

Captures d'écran :



Mon environnement : Visual Studio Code ouvert sur le projet InnovShop. On voit l'arborescence Symfony (`config`, `migrations`, `src`, `templates`, `tests`…), le fichier `.env.local` qui n'est pas envoyé sur GitHub, et la branche Git `main` en bas à gauche.



L'organisation du travail : le tableau Kanban Trello d'InnovShop (Backlog, À faire, En cours, Terminé). 45 user stories sont terminées.

2. Précisez les moyens utilisés :

- **Matériel :** Chromebook Acer 314, environnement Linux Debian 12.
- **Logiciels :** Visual Studio Code, PHP 8.2, Composer, MariaDB 10.11, Node.js et npm, Git, Symfony CLI.
- **Services en ligne :** GitHub (dépôts `LettreH/cabinet-dentaire` et `LettreH/innovshop`), Trello pour le suivi des tâches.
- **Documentation :** documentation officielle de Symfony, de PHP et de Git.

3. Avec qui avez-vous travaillé ?

J'ai installé et configuré mon environnement en autonomie, en suivant les consignes de la formation. Mon formateur et mon mentor m'ont aidé à faire les bons choix techniques, par exemple garder les outils déjà installés pour éviter les conflits. Pour comprendre les messages d'erreur et les commandes, j'ai consulté la documentation officielle.

4. Contexte

Nom de l'entreprise, organisme ou association	**Global Digital University**
Chantier, atelier, service	Mise en place de l'environnement de développement (installation de base du 09/03 au 20/03/2026, Symfony du 07/09 au 11/09/2026)
Période d'exercice	**Du** 09/03/2026 **au** 11/09/2026

5. Informations complémentaires *(facultatif)*

Cet environnement m'a servi pour tous mes projets de la formation : JavaScript (EM SERVICES), SQL (SoundWave), PHP orienté objet (cabinet dentaire) et Symfony (InnovShop).

Activité-type 1 — Développer la partie front-end d'une application web ou web mobile sécurisée

Exemple n°2 ▸ *Maquetter des interfaces utilisateur web ou web mobile*

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

Pour le projet **EM SERVICES**, une application de gestion de tâches pour une agence fictive, j'ai réalisé les maquettes des écrans avant d'écrire la moindre ligne de code.

1. Comprendre le besoin. J'ai lu le cahier des charges et listé ce que l'utilisateur doit pouvoir faire : se connecter, voir ses tâches, ouvrir le détail d'une tâche, la marquer comme terminée ou la supprimer, et consulter des statistiques.

2. Définir les écrans. J'en ai déduit 4 pages :

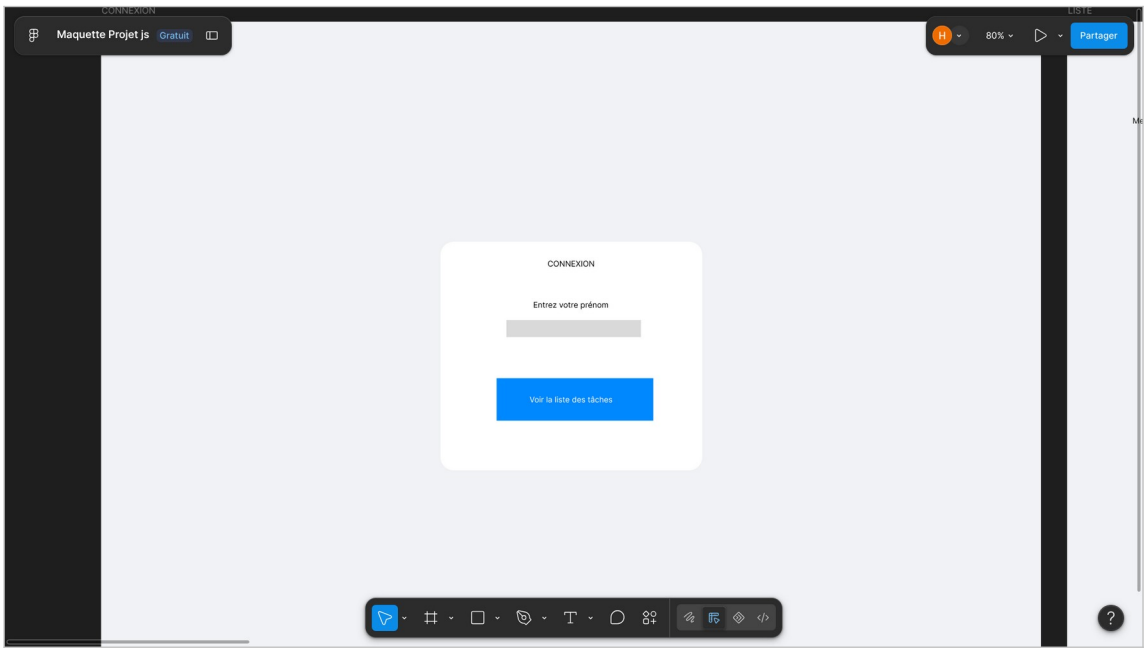
Page	Ce qu'elle contient
Connexion	Un champ pour entrer son prénom et un bouton « Voir la liste des tâches »
Liste des tâches	Un champ et un bouton « Ajouter », puis les tâches avec leurs détails
Détail d'une tâche	Le titre, l'ID, la date et les tags, avec les boutons « Terminé » et « Supprimer »
Statistiques	Le total, les tâches terminées et à faire, et une barre de progression

3. Dessiner les maquettes dans Figma. J'ai d'abord fait des maquettes fil de fer (*wireframes*) : des blocs gris qui montrent où se placent le menu, les titres, les listes et les boutons, sans couleurs ni images. Ce zoning permet de valider l'organisation de la page avant de travailler le design.

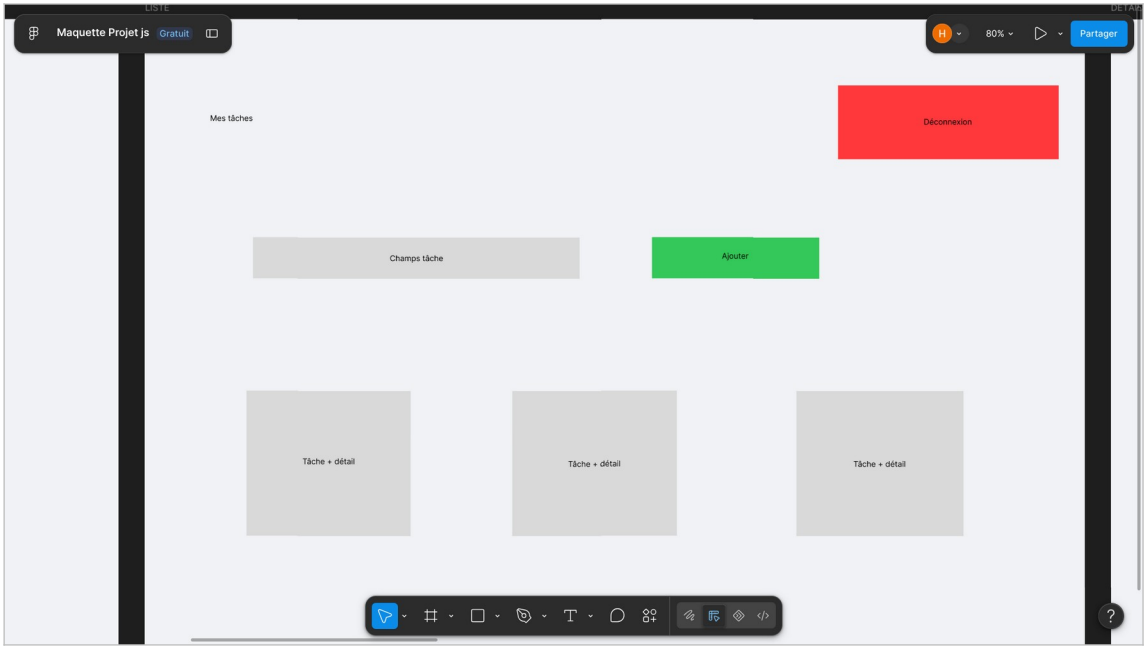
4. Penser au parcours de l'utilisateur. J'ai vérifié qu'on passe d'une page à l'autre de façon logique : connexion, puis liste, puis détail, avec un retour possible à la liste depuis chaque page.

5. Penser au responsive. J'ai dessiné chaque écran en deux versions, ordinateur et mobile. Sur téléphone, les éléments côte à côte passent les uns sous les autres, et les boutons « Retour » et « Déconnexion » restent toujours en haut.

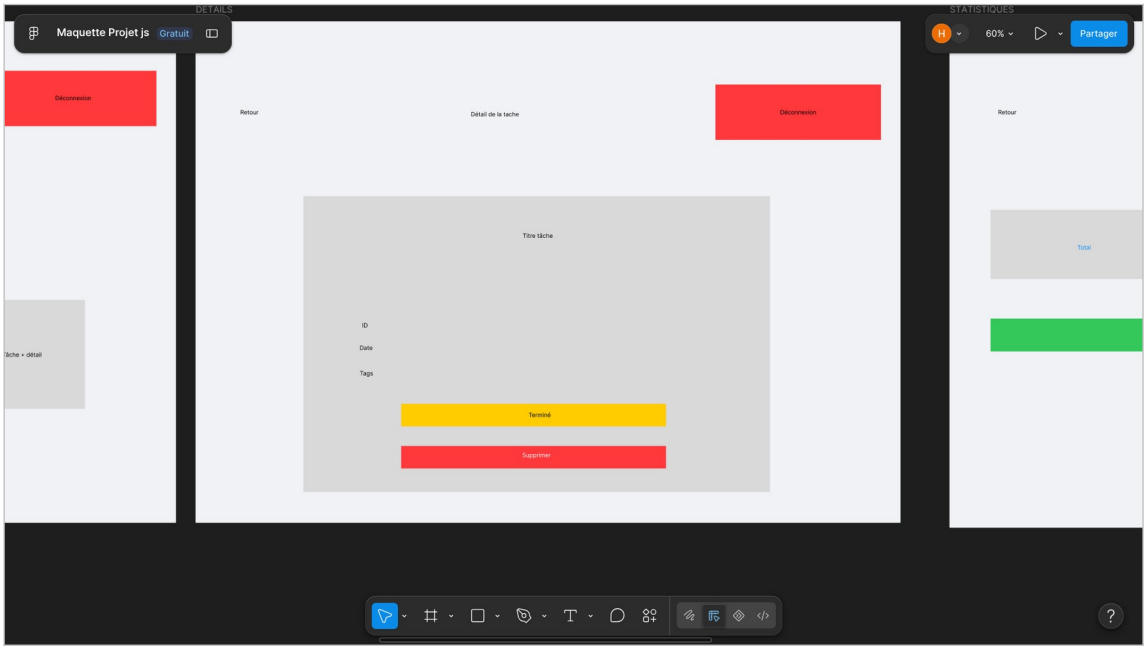
Mes maquettes fil de fer (Figma) :



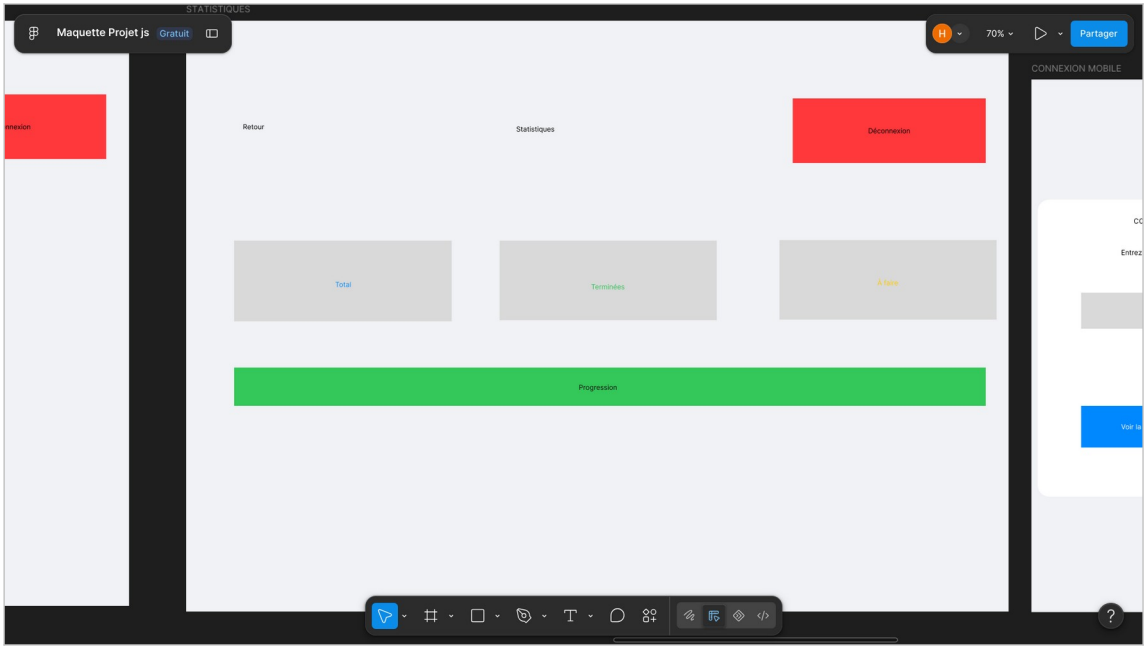
Connexion (ordinateur) : le visiteur entre son prénom.



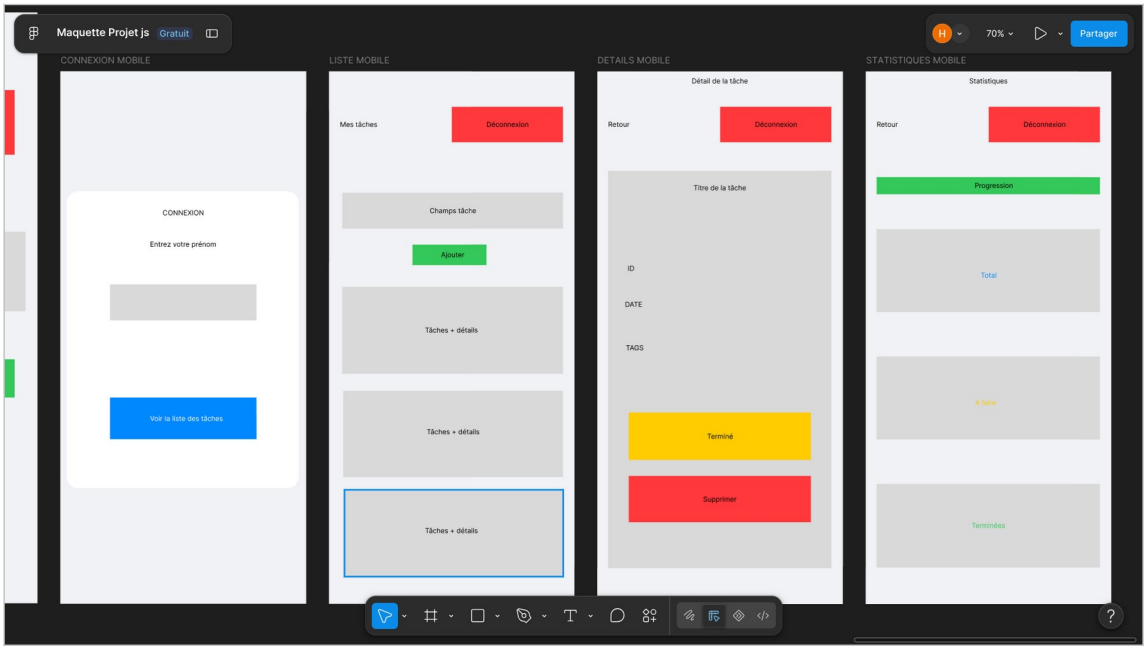
Liste des tâches (ordinateur) : le champ pour ajouter une tâche, puis les tâches.



Détail d'une tâche (ordinateur) : les informations et les boutons « Terminé » (jaune) et « Supprimer » (rouge).



Statistiques (ordinateur) : total, terminées, à faire, et la barre de progression.



Les 4 écrans en version mobile : les blocs s'empilent verticalement.

J'ai ensuite gardé cette méthode pour mes autres projets. Pour InnovShop, j'ai pensé chaque page en « mobile first » avec la grille de Bootstrap : 1 carte produit par ligne sur téléphone, 2 sur tablette, 3 sur ordinateur.

2. Précisez les moyens utilisés :

- **Figma**, pour les maquettes fil de fer.
- Le **cahier des charges** du projet EM SERVICES.
- Les **principes d'ergonomie** vus en formation : un bouton d'action bien visible, une navigation cohérente sur toutes les pages, des messages clairs pour l'utilisateur.

3. Avec qui avez-vous travaillé ?

J'ai réalisé les maquettes seul, puis je les ai présentées à mon formateur. Ses retours m'ont aidé à simplifier certains écrans avant de passer au code.

4. Contexte

Nom de l'entreprise, organisme ou association ▸ **Global Digital University**

Chantier, atelier, service ▸ Projet « EM SERVICES » — maquettes de l'application de gestion de tâches

Période d'exercice ▸ **Du** 04/05/2026 **au** 08/05/2026

5. Informations complémentaires *(facultatif)*

Faire les maquettes avant le code m'a évité de refaire des pages entières : les décisions sur l'organisation étaient déjà prises.

Activité-type 1 — Développer la partie front-end d'une application web ou web mobile sécurisée

Exemple n°3 ▸ *Réaliser des interfaces utilisateur statiques web ou web mobile*

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

Pour le site du **cabinet dentaire du Dr Dupont**, j'ai réalisé les pages publiques en HTML et CSS, sans framework, comme le demandait le cahier des charges.

Les pages réalisées : l'accueil (présentation du cabinet, liste des soins, horaires d'ouverture), la liste des actualités, le détail d'une actualité, et le formulaire de prise de rendez-vous.

1. Un HTML structuré et sémantique. J'ai utilisé les balises qui ont un sens : `header`, `nav`, `main`, `section`, `article` et `footer`. Chaque page a un seul titre `h1`, puis des `h2` pour les parties. Le code est plus lisible, et le site est mieux compris par les moteurs de recherche et les lecteurs d'écran.

2. Des morceaux réutilisables. L'en-tête avec le menu et le pied de page sont écrits une seule fois, dans des fichiers « partials » inclus sur chaque page. Pour changer un lien du menu, je modifie un seul fichier.

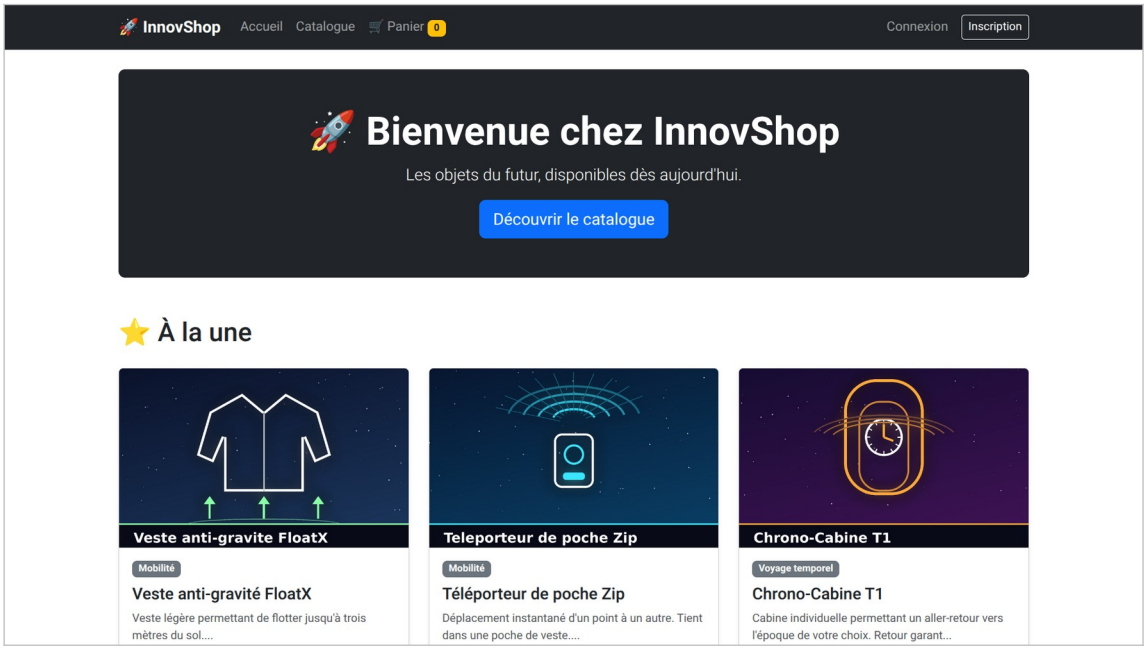
3. Un CSS responsive. J'ai écrit le CSS en « mobile first » : d'abord la version téléphone, puis des *media queries* qui adaptent la mise en page aux écrans plus larges. Les blocs empilés sur téléphone se placent côte à côte sur ordinateur grâce à Flexbox.

4. L'accessibilité. Chaque image a un texte alternatif (`alt`), chaque champ du formulaire a un `label` relié, et les couleurs gardent un bon contraste.

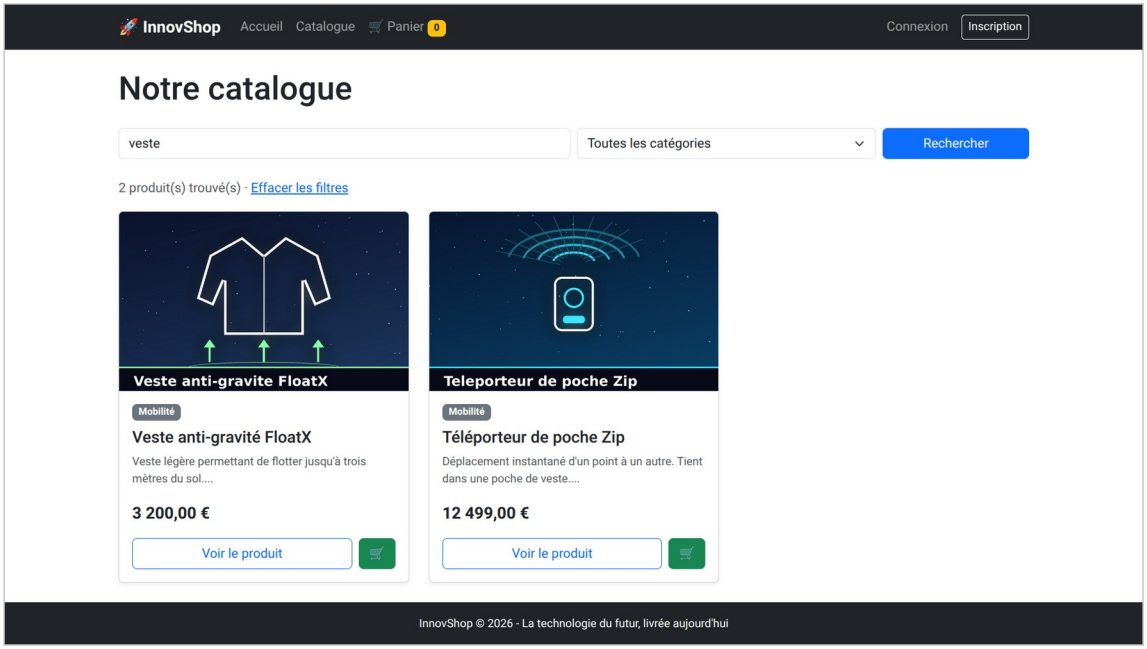
5. Les tests d'affichage. J'ai testé chaque page avec le mode « appareils » de Chrome : téléphone, tablette et ordinateur.

Pour **InnovShop**, j'ai utilisé le framework CSS **Bootstrap 5** avec des templates **Twig**. Toutes les pages héritent d'un gabarit commun (`base.html.twig`), et la carte produit est un morceau réutilisable (`_carte.html.twig`), affiché sur l'accueil et dans le catalogue. L'audit **Lighthouse** du site en ligne donne 100 en accessibilité.

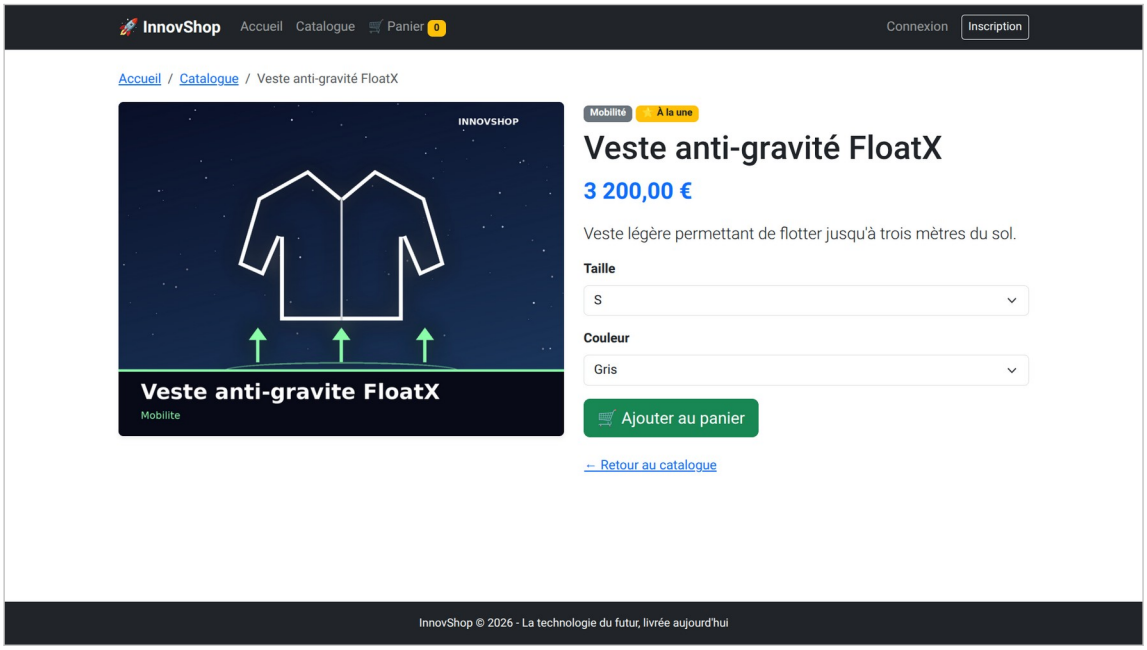
Captures d'écran (InnovShop, site en ligne) :



Page d'accueil : la bannière et les produits « à la une », affichés avec le même morceau de template `_carte.html.twig`.



Catalogue : recherche « veste », 2 produits trouvés, avec le filtre par catégorie.



Fiche produit : prix, description et choix des options (taille, couleur).

2. Précisez les moyens utilisés :

- **Langages :** HTML5, CSS3 (Flexbox, media queries).
- **Pour InnovShop :** Bootstrap 5.3, Twig.
- **Outils :** Visual Studio Code, les outils de développement de Chrome (mode responsive, Lighthouse).

3. Avec qui avez-vous travaillé ?

J'ai réalisé les pages en autonomie, à partir du cahier des charges. Mon formateur a relu mon travail et m'a conseillé d'organiser les vues en partials.

4. Contexte

Nom de l'entreprise, organisme ou association	**Global Digital University**
Chantier, atelier, service	Projet « Cabinet dentaire du Dr Dupont » — front office public
Période d'exercice	**Du** 30/07/2026 **au** 16/08/2026

5. Informations complémentaires *(facultatif)*

Site en ligne : https://hermann.alwaysdata.net — Code source : https://github.com/LettreH/cabinet-dentaire

Activité-type 1

Développer la partie front-end d'une application web ou web mobile sécurisée

Exemple n°4 *Développer la partie dynamique des interfaces utilisateur web ou web mobile*

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

Exemple principal : EM SERVICES, une application en JavaScript reliée à une API.

L'application de gestion de tâches communique avec une **API REST** : un serveur qui renvoie et enregistre des données au format JSON. J'ai développé le JavaScript des 4 pages.

1. **Récupérer et afficher les tâches.** Avec `fetch()` et `.then()`, j'envoie une requête `GET` à l'API, je lis la réponse JSON, puis je crée les éléments HTML de la liste avec le DOM.
2. **Modifier et supprimer.** J'envoie une requête `PUT` pour modifier une tâche, et `DELETE` pour la supprimer. La page se met à jour sans être rechargée.
3. **Garder des informations dans le navigateur.** J'ai utilisé le `localStorage` pour mémoriser des données entre deux pages, par exemple l'utilisateur connecté.
4. **Une page de statistiques dynamique.** Le script compte les tâches par statut et affiche les chiffres.
5. **Les tests.** J'ai écrit des tests unitaires avec **Jest** pour vérifier mes fonctions.

Les bugs que j'ai corrigés :

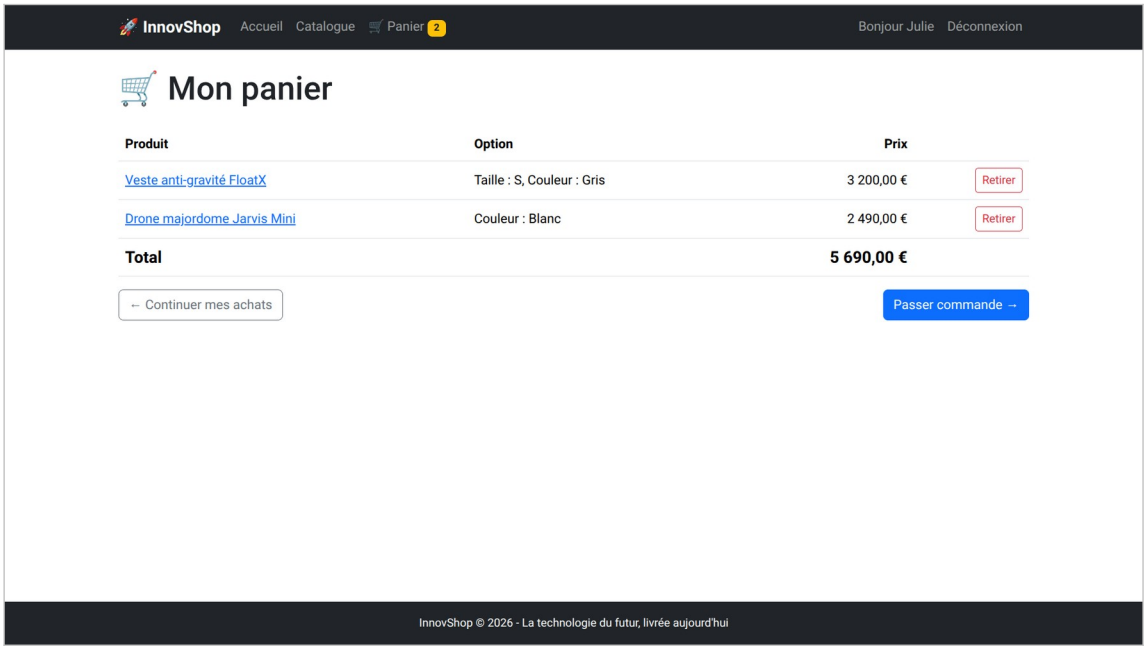
Bug	Cause	Correction
Une fonction ne faisait pas ce que j'attendais	Elle était définie deux fois, la deuxième écrasait la première	Suppression du doublon
Erreur quand un champ manquait	La réponse de l'API ne contenait pas toujours ce champ	Une valeur par défaut (opérateur `??` ou `||`)
Erreur après une suppression	La réponse à un `DELETE` n'est pas du JSON	Ne lire le JSON que s'il y en a
Mauvais statut affiché	Une erreur dans la condition d'affichage	Correction de la logique

Deuxième exemple : le panier en AJAX d'InnovShop.

Quand le client clique sur « Ajouter au panier » ou « Retirer », mon script bloque le rechargement de la page (`preventDefault`), envoie le formulaire au serveur avec `fetch`, et reçoit une réponse JSON (nombre d'articles, total). Il met alors à jour le compteur 🛒 et le total. En cas d'erreur, un `try/catch` affiche un message rouge à l'utilisateur au lieu de ne rien faire.

La sécurité côté navigateur : j'affiche le texte avec `textContent`, et non `innerHTML`. Si un texte contient du code, il s'affiche comme du texte et n'est pas exécuté : c'est une protection contre le XSS. Le jeton CSRF des formulaires est envoyé avec chaque requête AJAX.

Capture d'écran (InnovShop) :



Le panier : chaque ajout crée une ligne avec son option. Le compteur 🛒 en haut (2) se met à jour en AJAX, sans recharger la page.

2. Précisez les moyens utilisés :

- **Langage :** JavaScript (ES6+) : `fetch`, `async/await`, manipulation du DOM, `localStorage`, `try/catch`.
- **Tests :** Jest.
- **Outils :** Visual Studio Code, la console et l'onglet Réseau des outils de développement de Chrome.
- **Mise en ligne d'EM SERVICES :** l'API sur Vercel, le front-end sur GitHub Pages.

3. Avec qui avez-vous travaillé ?

J'ai développé ces fonctionnalités seul. J'ai utilisé l'onglet Réseau de Chrome pour comprendre ce que l'API renvoyait, et mon formateur m'a guidé sur la gestion des erreurs.

4. Contexte

Nom de l'entreprise, organisme ou association ▸ **Global Digital University**

Chantier, atelier, service ▸ Projet « EM SERVICES » (du 11/05 au 05/06/2026), puis panier du projet « InnovShop » (du 21/09 au 25/09/2026)

Période d'exercice ▸ **Du** 11/05/2026 **au** 25/09/2026

5. Informations complémentaires *(facultatif)*

Le panier d'InnovShop fonctionne aussi sans JavaScript, avec un rechargement normal de la page : c'est l'amélioration progressive.

Activité-type 2 — Développer la partie back-end d'une application web ou web mobile sécurisée

Exemple n°1 ▸ *Mettre en place une base de données relationnelle*

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

Exemple principal : la base du cabinet dentaire.

1. Concevoir. À partir du cahier des charges, j'ai repéré les données à stocker et leurs liens, puis j'ai dessiné le modèle. J'ai obtenu 6 tables :

Table	Contenu	Liens
`utilisateurs`	Le Dr Dupont et son équipe (back office)	Aucun
`patients`	Les patients du cabinet	Aucun
`services`	Les soins proposés, leur durée et leur prix	Aucun
`horaires`	Les horaires d'ouverture, un jour par ligne	Aucun
`actualites`	Les articles publiés sur le site	Vers `utilisateurs` (l'auteur)
`rendez_vous`	Les demandes de rendez-vous	Vers `patients` et vers `services`

2. Écrire le script SQL. J'ai créé la base en `utf8mb4` (pour gérer les accents) et chaque table avec le moteur InnoDB, qui gère les clés étrangères. J'ai créé les tables dans le bon ordre : d'abord celles qui ne dépendent d'aucune autre, et `rendez_vous` en dernier.

3. Choisir les bons types et les contraintes.

- Une clé primaire `id` en `AUTO_INCREMENT` dans chaque table.
- `NOT NULL` sur les champs obligatoires, `UNIQUE` sur les emails.
- Des types adaptés : `DECIMAL(8,2)` pour les prix, `DATETIME` pour les rendez-vous, `ENUM` pour un statut qui a peu de valeurs possibles.
- Un champ `mot_de_passe` en `VARCHAR(255)`, car le mot de passe y est stocké **haché**, jamais en clair.

4. Choisir ce qui se passe à la suppression (les clés étrangères) :

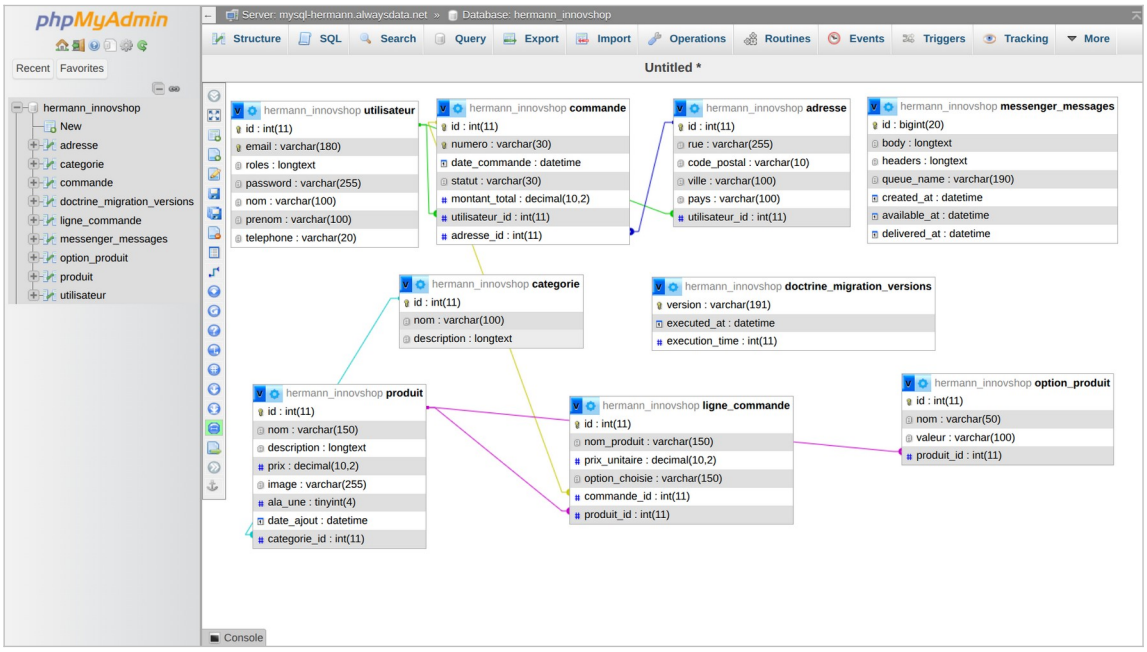
Règle	Exemple	Pourquoi
`ON DELETE CASCADE`	Supprimer un patient supprime ses rendez-vous	Un rendez-vous sans patient n'a plus de sens
`ON DELETE RESTRICT`	Impossible de supprimer un soin encore utilisé	On protège les rendez-vous existants
`ON DELETE SET NULL`	Un auteur qui part : ses articles restent, sans auteur	Les articles restent en ligne

5. Optimiser. J'ai ajouté des index sur les colonnes utilisées pour chercher ou trier : le nom du patient, la date et le statut des rendez-vous.

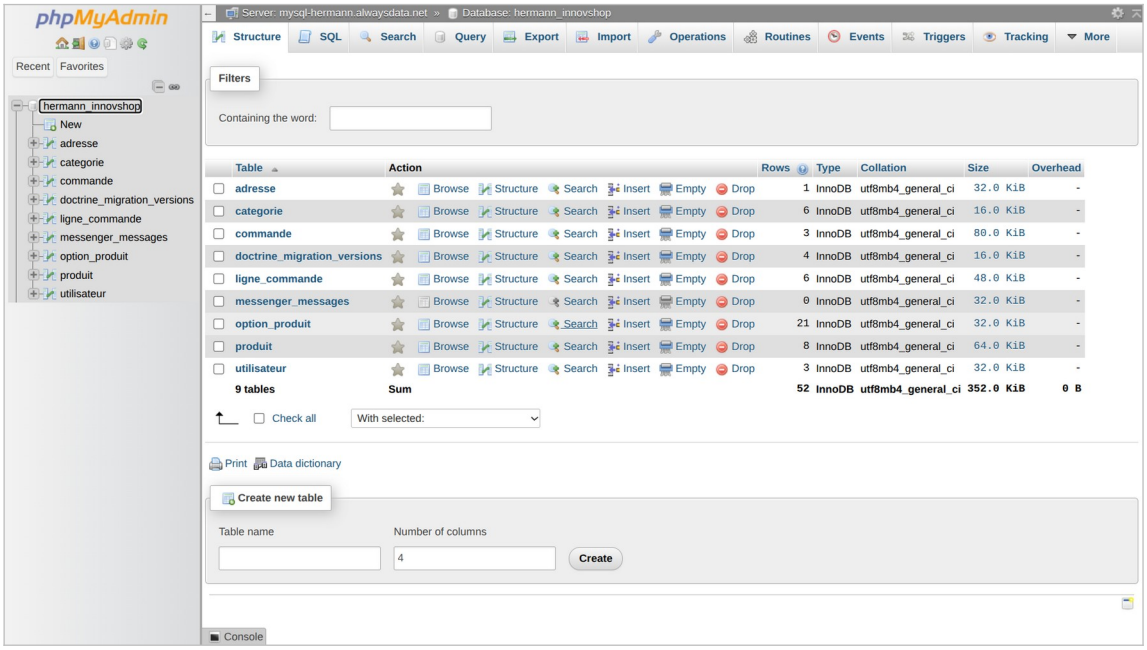
6. Remplir et tester. J'ai écrit un script d'insertion de données de test, puis vérifié le résultat avec `SHOW TABLES` et des `SELECT`.

Exemple précédent : SoundWave, une base pour une société d'événements musicaux. J'ai réalisé le diagramme entité-association dans Draw.io, le script de création, le script d'insertion, et des requêtes avec `JOIN`, `GROUP BY`, `SUM`, `COUNT(DISTINCT)` et `HAVING`. Une erreur de doublon bloquée par une contrainte `UNIQUE` m'a montré l'utilité des contraintes : la base protège elle-même la qualité des données.

Troisième exemple : la base d'InnovShop. Ici, je n'ai pas écrit le SQL à la main : les tables sont créées par les **migrations Doctrine**, à partir de mes entités PHP. Voici la base en ligne, vue dans phpMyAdmin.



Le schéma de la base dans le Concepteur de phpMyAdmin : les tables et les liens entre elles (clés étrangères `utilisateur_id`, `adresse_id`, `commande_id`, `produit_id`, `categorie_id`).



Les 9 tables de la base `hermann_innovshop`, en InnoDB et en `utf8mb4`. Les tables `doctrine_migration_versions` et `messenger_messages` sont créées par Symfony.

2. Précisez les moyens utilisés :

- **SGBD :** MariaDB, utilisé dans le terminal Linux.
- **Conception :** Draw.io (diagramme entité-association, diagramme de classes).
- **Langage :** SQL (création, insertion, requêtes).

3. Avec qui avez-vous travaillé ?

J'ai conçu et créé les bases seul. Mon formateur a validé mon modèle avant que je passe au code PHP.

4. Contexte

Nom de l'entreprise, organisme ou association ▸ **Global Digital University**

Chantier, atelier, service ▸ Projet « SoundWave » (du 15/06 au 26/06/2026), puis base du projet « Cabinet dentaire » (du 30/07 au 02/08/2026)

Période d'exercice ▸ **Du** 15/06/2026 **au** 02/08/2026

5. Informations complémentaires *(facultatif)*

Dans InnovShop, j'ai fait évoluer la base avec les migrations Doctrine : chaque changement de structure est un fichier, rejoué dans le même ordre sur mon ordinateur et sur le serveur.

Activité-type 2 — Développer la partie back-end d'une application web ou web mobile sécurisée

Exemple n°2 ▸ *Développer des composants d'accès aux données SQL et NoSQL*

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

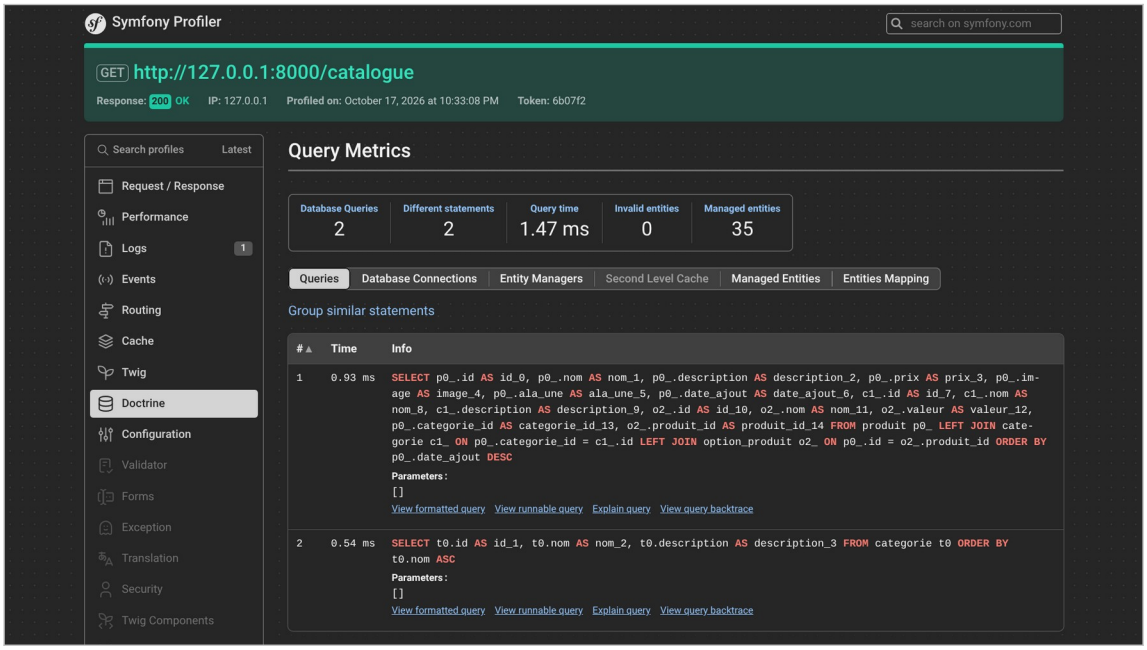
Exemple 1 : le cabinet dentaire, en PHP orienté objet avec PDO (sans framework).

1. **Une classe de connexion.** J'ai écrit une classe `Database` qui se connecte à MariaDB avec **PDO**. Ses attributs sont privés, et les identifiants sont lus dans un fichier `config.php` qui n'est pas envoyé sur GitHub. La connexion n'est créée qu'une seule fois, puis réutilisée. J'ai activé le mode `ERRMODE_EXCEPTION`, pour que les erreurs SQL soient signalées clairement.
2. **La séparation Entité / Manager.** Sur les conseils de mon formateur, j'ai séparé chaque donnée en deux classes : l'**entité** (par exemple `Patient`) représente une ligne de la table, et le **Manager** (`PatientManager`) contient les requêtes : lister, trouver, ajouter, modifier, supprimer. J'ai fait de même pour les services, les actualités, les horaires et les rendez-vous.
3. **L'hydratation.** Une méthode remplit automatiquement un objet à partir d'une ligne renvoyée par la base. Le reste du code manipule des objets, pas des tableaux.
4. **Les requêtes préparées.** Toutes mes requêtes utilisent `prepare()` puis `execute()` avec des paramètres. Une donnée tapée par l'utilisateur n'est jamais collée dans le SQL : c'est la protection contre l'injection SQL.
5. **Une méthode métier : `trouverOuCreer`.** Quand un visiteur demande un rendez-vous, je cherche le patient par son email ; s'il n'existe pas, je crée sa fiche. Puis j'enregistre le rendez-vous avec le statut « en attente ».
6. **Le chargement automatique des classes** avec `spl_autoload_register`, à la place d'une longue liste de `require_once`.

Exemple 2 : InnovShop, avec l'ORM Doctrine (Symfony).

1. Les **entités** PHP (`Produit`, `Commande`…) décrivent les tables et leurs relations avec des attributs, comme `#[ORM\ManyToOne]`.
2. Les **Repositories** contiennent les requêtes. J'ai écrit `findDerniers()`, `findALaUne()` et `rechercher()` avec le **QueryBuilder** et `setParameter()`.
3. **L'optimisation.** Avec le Profiler de Symfony, j'ai repéré un problème « N+1 » : une requête pour la liste des produits, puis une requête de plus pour les options de chaque produit. Je l'ai corrigé avec une jointure (`leftJoin` et `addSelect`) : le catalogue est passé de **10 à 2 requêtes**. J'ai aussi ajouté des index sur les colonnes de tri.

Capture d'écran (Symfony Profiler, onglet Doctrine) :



*Après la correction du N+1, la page catalogue ne fait plus que **2 requêtes**. La première récupère les produits avec leur catégorie et leurs options grâce aux `LEFT JOIN`. La seconde charge les catégories du filtre.*

Le NoSQL : mes projets utilisent des bases relationnelles, adaptées à des données très liées entre elles (clients, commandes, produits). Lors d'un workshop de la formation, j'ai **installé MongoDB**, une base NoSQL orientée documents, mais je ne l'ai pas encore utilisée dans un projet. J'en connais le principe : les données sont rangées dans des **documents JSON**, sans schéma fixe et sans jointures. C'est pratique pour des données peu structurées ou très nombreuses. Pour InnovShop, une base relationnelle était le meilleur choix, car une commande est liée à un client, à une adresse et à des produits.

2. Précisez les moyens utilisés :

- **Cabinet dentaire :** PHP 8.2 orienté objet, PDO, MariaDB.
- **InnovShop :** Symfony 7.4, Doctrine ORM, QueryBuilder, Symfony Profiler.
- **Tests :** 36 tests unitaires PHPUnit sur les entités du cabinet dentaire.

3. Avec qui avez-vous travaillé ?

J'ai développé ces composants seul. Mon formateur a relu mon code du cabinet dentaire et m'a demandé d'ajouter l'autoload, la séparation Entité / Manager et l'hydratation : j'ai appliqué ces trois améliorations.

4. Contexte

Nom de l'entreprise, organisme ou association ▸ **Global Digital University**

Chantier, atelier, service ▸ Projets « Cabinet dentaire du Dr Dupont » (du 02/08 au 24/08/2026) et « InnovShop » (du 07/09 au 08/10/2026)

Période d'exercice ▸ **Du** 02/08/2026 **au** 08/10/2026

5. Informations complémentaires *(facultatif)*

Faire d'abord le cabinet dentaire sans framework m'a aidé à comprendre ce que Doctrine fait à ma place dans InnovShop : la connexion, les requêtes préparées et l'hydratation des objets.

Activité-type 2 — Développer la partie back-end d'une application web ou web mobile sécurisée

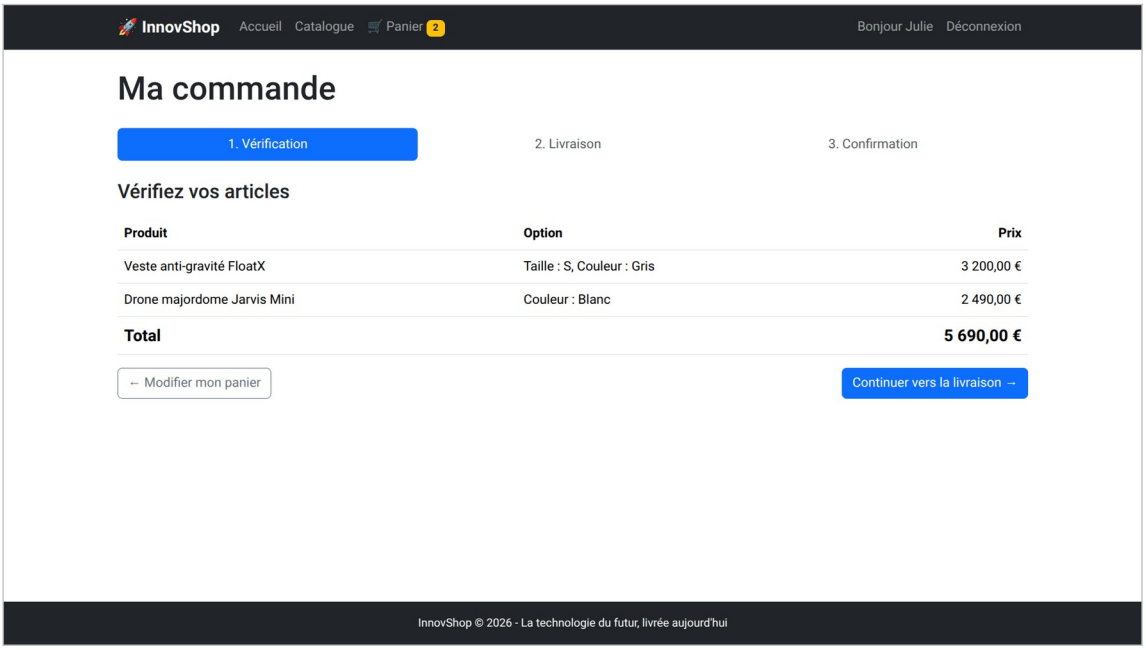
Exemple n°3 ▸ *Développer des composants métier côté serveur*

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

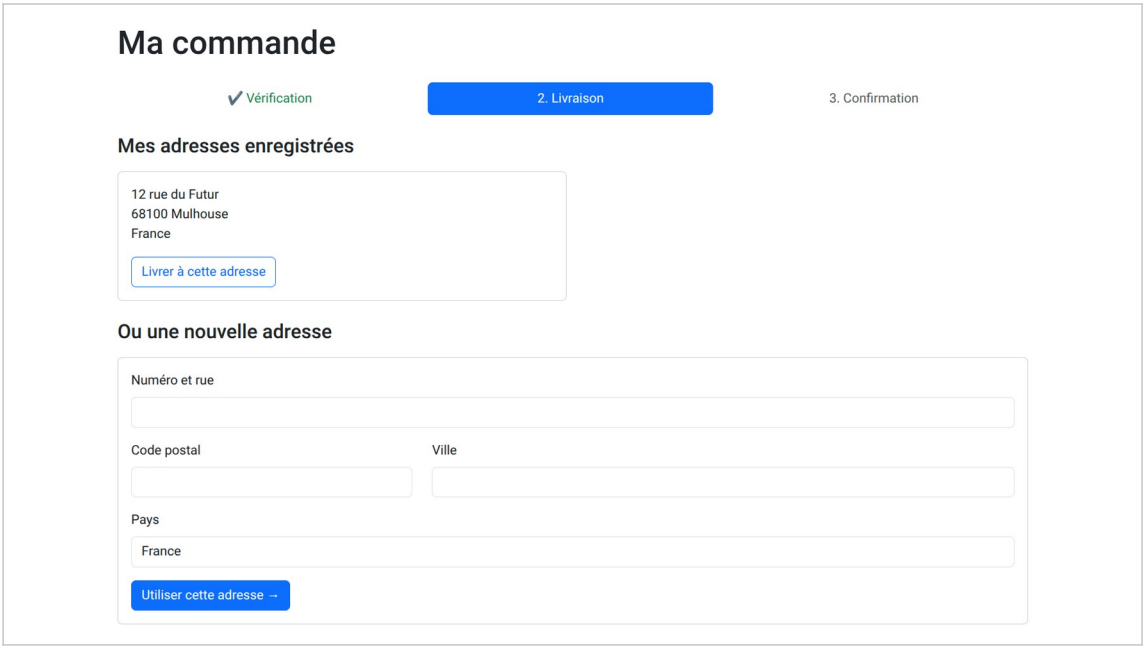
Un **composant métier**, c'est le code qui applique les règles de la boutique de Madame Leroy. Dans InnovShop, j'ai gardé des **controllers légers** et j'ai placé la logique dans des **services**.

1. **Le panier (`PanierService`).** Le panier n'est pas une table : il est rangé dans la **session** du visiteur. Le service sait ajouter un produit avec son option, supprimer une ligne, vider le panier, calculer le total et compter les articles. Le controller l'appelle et répond en **JSON** quand la demande vient d'AJAX, pour que le compteur 🛒 se mette à jour sans recharger la page.
2. **La validation de la commande (`CommandeController::valider`).** Quand le client confirme :
   - je vérifie que l'adresse lui appartient (Voter) ;
   - je crée la commande avec un **numéro unique** (`CMD-20261005-A1B2C3`) ;
   - je **copie** le nom et le prix de chaque produit dans les lignes de commande, pour que la facture ne change pas si le prix change plus tard ;
   - j'enregistre tout avec Doctrine, je vide le panier et j'envoie un **email de confirmation** (Symfony Mailer et un template Twig).
3. **L'email automatique au changement de statut (`CommandeStatutListener`).** Quand l'administrateur passe une commande de « validée » à « expédiée », un **écouteur d'événements Doctrine** le détecte et envoie un email au client. Je n'ai pas eu à modifier le back-office.
4. **La facture PDF (`FactureService`).** Le service transforme un template Twig en PDF avec **Dompdf**. Le client télécharge sa facture depuis son espace.
5. **Les règles d'accès (Voters).** `CommandeVoter` : seul le propriétaire de la commande (ou un admin) peut la voir. `AdresseVoter` : seul le propriétaire peut modifier son adresse. Cela empêche un client de voir la commande d'un autre en changeant le numéro dans l'adresse de la page.
6. **La validation des données.** Les formulaires (`AdresseType`, `ProfilType`, `ChangerMotDePasseType`) sont vérifiés côté serveur avec des contraintes `#[Assert\…]`. Le nouveau mot de passe est haché en **bcrypt**.
7. **Les tests.** J'ai écrit **17 tests PHPUnit** : des tests unitaires (calcul des quantités, valeurs par défaut) et des tests fonctionnels (pages publiques, redirection vers la connexion, accès admin refusé au client, Voter). Résultat : **OK (17 tests, 35 assertions)**.

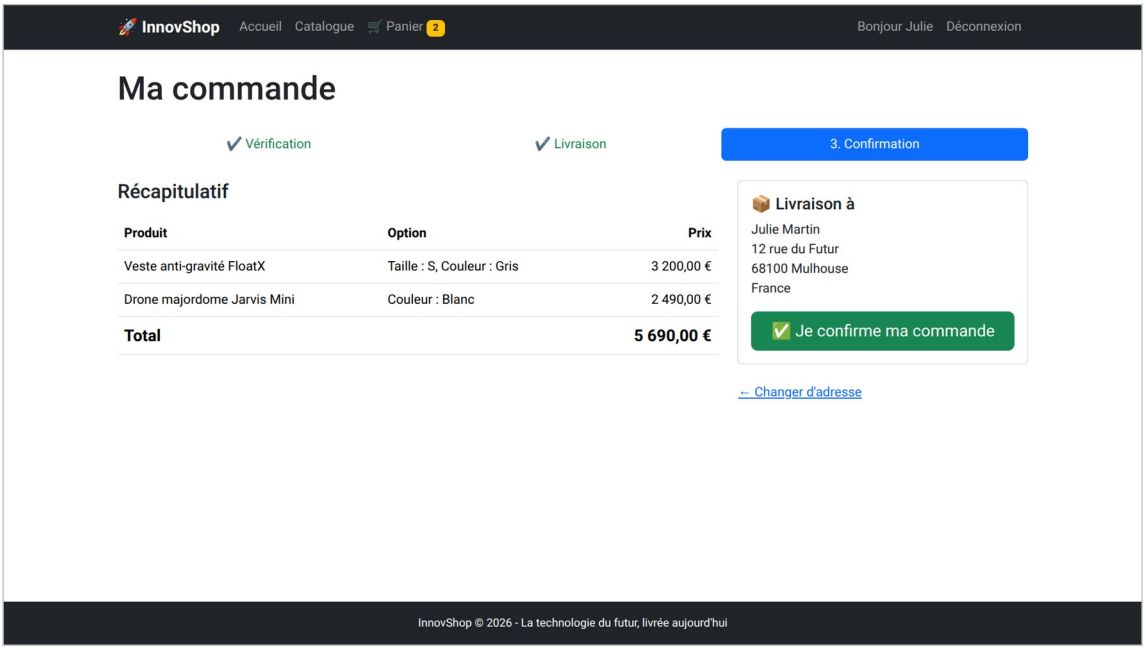
Captures d'écran (InnovShop, site en ligne) :



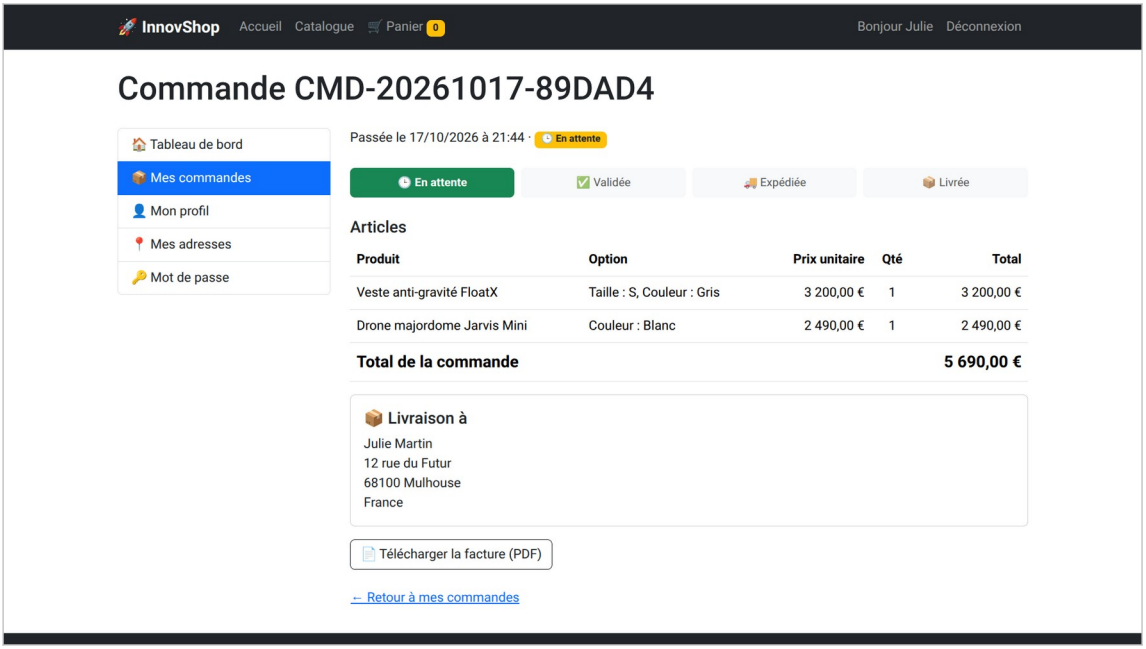
La commande en 3 étapes. Étape 1 : le client vérifie ses articles.



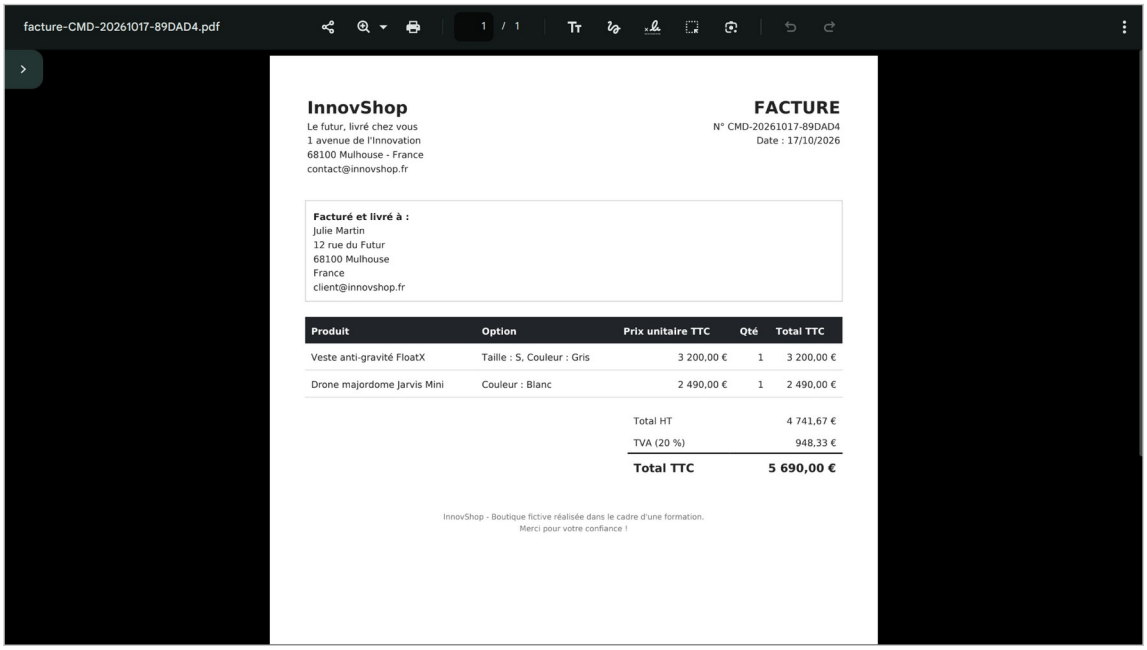
Étape 2 : il choisit une adresse enregistrée ou en saisit une nouvelle (formulaire `AdresseType` validé côté serveur).



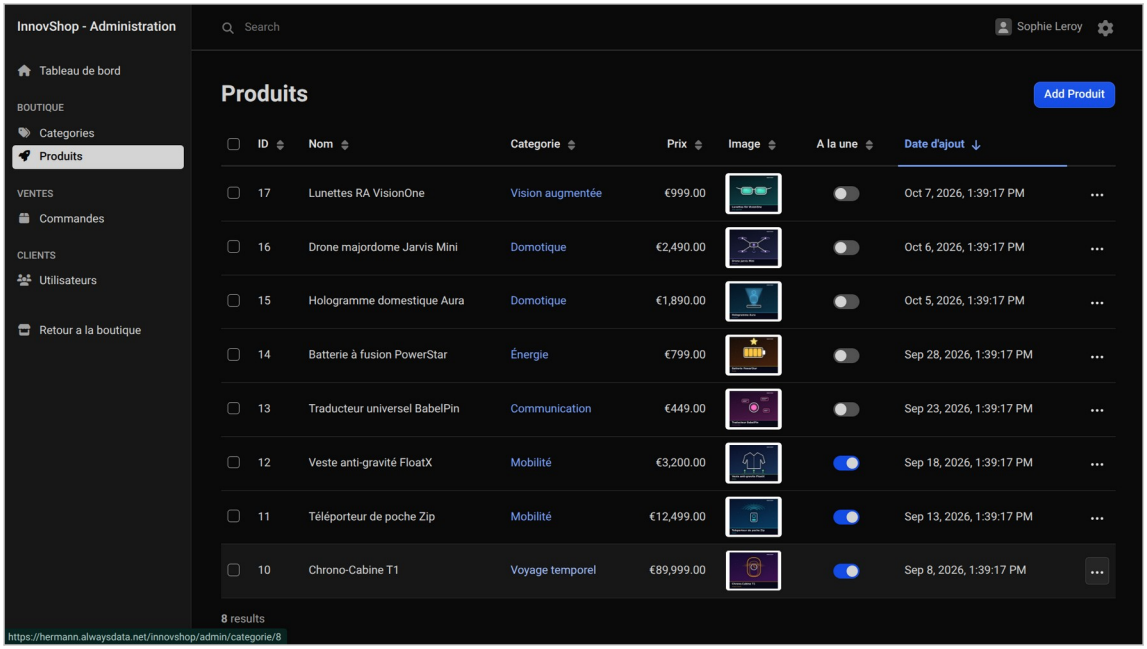
Étape 3 : le récapitulatif. Le bouton « Je confirme ma commande » appelle `CommandeController::valider`.



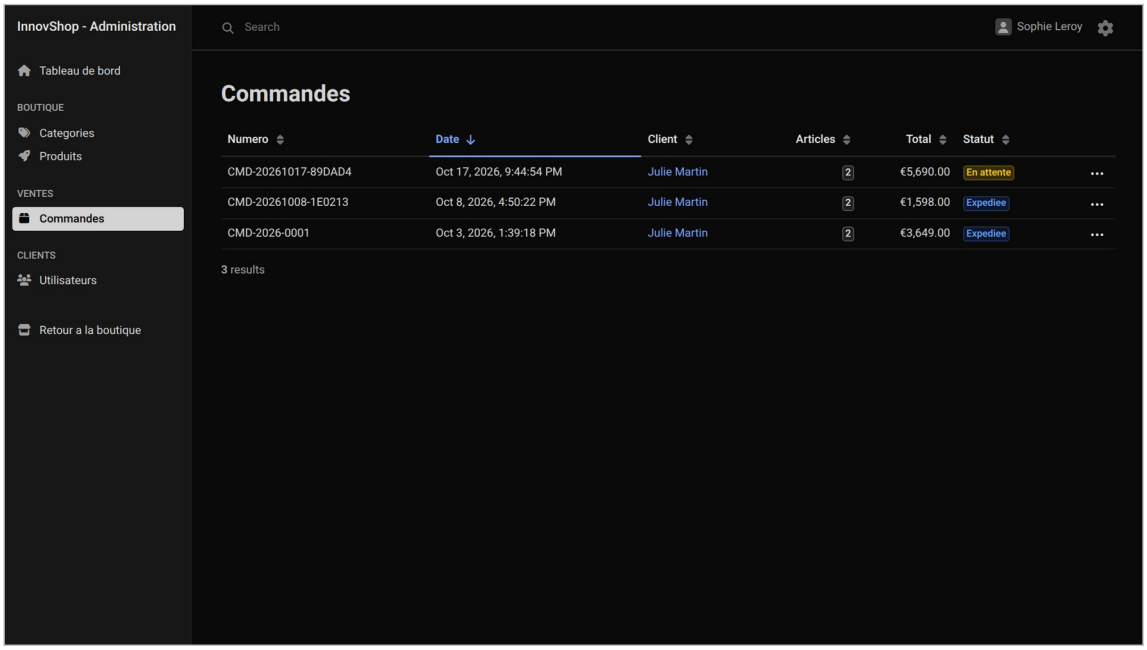
Espace client : le détail de la commande avec son numéro unique, le suivi du statut et le bouton de la facture. Seul le propriétaire peut ouvrir cette page (`CommandeVoter`).



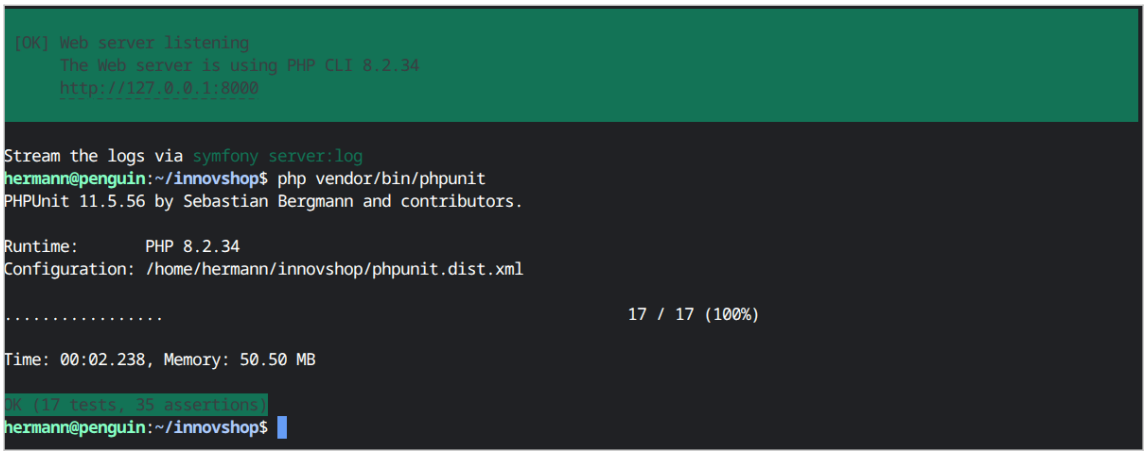
La facture PDF générée par `FactureService` avec Dompdf : total HT, TVA 20 % et total TTC.



Back-office EasyAdmin : la liste des produits (image, catégorie, « à la une »).



Back-office : la liste des commandes et leur statut. Quand l'admin change un statut, `CommandeStatutListener` envoie un email au client.



*Les tests automatisés : `php vendor/bin/phpunit` donne **OK (17 tests, 35 assertions)**.*

2. Précisez les moyens utilisés :

- Symfony 7.4 : services, injection de dépendances, Voters, formulaires, Security.
- Symfony Mailer, Doctrine Entity Listener, Dompdf.
- PHPUnit avec une base de test séparée (`innovshop_test`).
- PHP CS Fixer pour respecter la norme **PSR-12**.

3. Avec qui avez-vous travaillé ?

J'ai développé ces composants seul, en suivant les consignes des séquences de la formation. Pour les notions nouvelles (Voters, écouteurs d'événements), je me suis appuyé sur la documentation officielle de Symfony ; j'ai testé et compris chaque partie du code.

4. Contexte

Nom de l'entreprise, organisme ou association	**Global Digital University**
Chantier, atelier, service	Projet « InnovShop » (cliente fictive : Madame Leroy)
Période d'exercice	**Du** 07/09/2026 **au** 08/10/2026

5. Informations complémentaires *(facultatif)*

Le test le plus utile : vérifier qu'un client connecté ne peut pas ouvrir la facture d'un autre client. Le Voter renvoie bien une erreur **403**.

Activité-type 2

Développer la partie back-end d'une application web ou web mobile sécurisée

Exemple n°4 ▸ *Documenter le déploiement d'une application dynamique web ou web mobile*

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

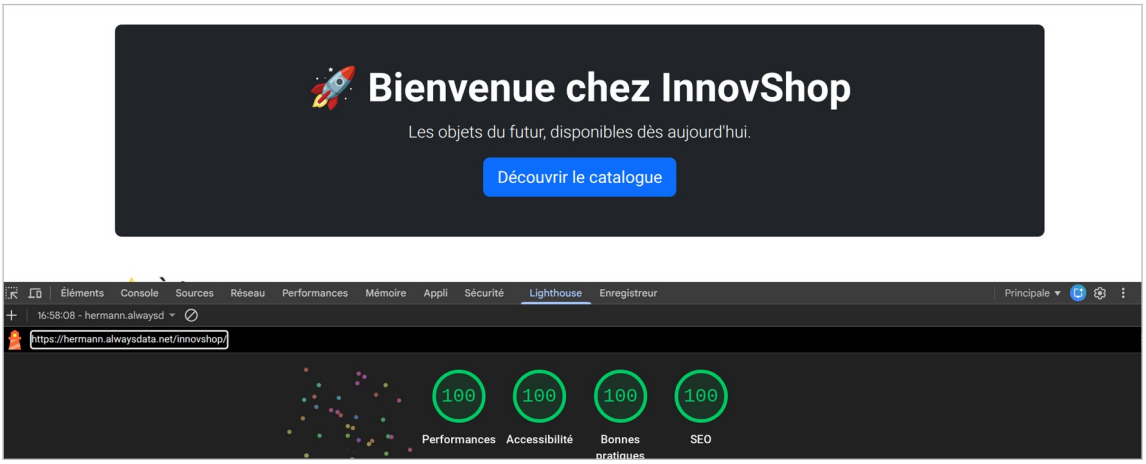
J'ai mis InnovShop en ligne chez l'hébergeur français **Alwaysdata**, puis j'ai écrit la procédure dans le **README** du dépôt GitHub, pour qu'une autre personne puisse la refaire.

1. **Préparer l'hébergement.** J'ai créé une base MariaDB et un site PHP 8.4. J'ai réglé le **répertoire racine sur** `public/` : seul ce dossier est visible depuis Internet, le code et les secrets restent protégés.
2. **Récupérer le code.** En **SSH**, j'ai cloné le dépôt GitHub sur le serveur avec `git clone`.
3. **Configurer la production.** J'ai créé un fichier `.env.local` sur le serveur, jamais envoyé sur GitHub, avec `APP_ENV=prod`, `APP_DEBUG=0`, un `APP_SECRET`, la `DATABASE_URL` et la configuration des emails.
4. **Installer et construire.**
   - `composer install --no-dev --optimize-autoloader` : seulement les bibliothèques utiles en production ;
   - `doctrine:migrations:migrate` : crée les tables ;
   - `asset-map:compile` : génère les fichiers CSS et JS minifiés ;
   - `cache:clear` : vide le cache.
5. **Les emails.** En ligne, les emails partent vers une boîte de test **Mailtrap** (SMTP), car les comptes de démonstration ont des adresses fictives.
6. **Vérifier.** J'ai testé tout le parcours en ligne (catalogue, panier, commande, facture, back-office) et les pages d'erreur. J'ai mesuré la qualité avec **Lighthouse** : **100 / 100 / 100 / 100**.
7. **Documenter la mise à jour.** Dans le README, j'ai noté les commandes pour mettre le site à jour : `git pull`, `composer install`, migrations, compilation des assets, vidage du cache.

J'ai rencontré et résolu plusieurs problèmes :

- le sous-domaine était refusé, donc j'ai publié le site dans un chemin : `hermann.alwaysdata.net/innovshop` ;
- le mot de passe de la base contenait un `@`, qui casse l'adresse de connexion : je l'ai encodé en `%40` ;
- le répertoire racine était mal renseigné (erreur « Not Found ») : je l'ai corrigé.

Capture d'écran :



L'audit Lighthouse du site en production (`https://hermann.alwaysdata.net/innovshop/`) : 100 en performances, accessibilité, bonnes pratiques et SEO.

2. Précisez les moyens utilisés :

- Alwaysdata : Apache, PHP 8.4, MariaDB 11.4, accès SSH.
- Git et GitHub, Composer, console Symfony.
- Mailtrap, Lighthouse.
- README en Markdown.

3. Avec qui avez-vous travaillé ?

J'ai réalisé le déploiement seul. J'ai envoyé les liens et les livrables à mon formateur pour l'évaluation.

4. Contexte

Nom de l'entreprise, organisme ou association	**Global Digital University**
Chantier, atelier, service	Projet « InnovShop » — mise en ligne sur Alwaysdata
Période d'exercice	**Du** 08/10/2026 **au** 16/10/2026

5. Informations complémentaires *(facultatif)*

- Site en ligne : https://hermann.alwaysdata.net/innovshop/
- Code et README : https://github.com/LettreH/innovshop

Titres, diplômes, CQP, attestations de formation

(facultatif)

Intitulé	Autorité ou organisme	Date
BTS Management des Unités Commerciales (MUC)	Ministère de l'Éducation nationale — obtenu à Luisant	2011
Formation Développeur Web et Web Mobile	Global Digital University	2026

Déclaration sur l'honneur

Je soussigné(e) [prénom et nom] ***Hermann MBALA*** ,

déclare sur l'honneur que les renseignements fournis dans ce dossier sont exacts et que je suis l'auteur(e) des réalisations jointes.

Fait à ***Fleury-Mérogis*** le ***31/10/2026***

pour faire valoir ce que de droit.

Signature : ***Hermann MBALA***

Documents illustrant la pratique professionnelle

(facultatif)

Intitulé
CP 1 — Visual Studio Code ouvert sur le projet InnovShop et tableau Kanban Trello du projet
CP 2 — Maquettes fil de fer Figma de l'application EM SERVICES, versions ordinateur et mobile
CP 3 — Captures du site InnovShop en ligne : accueil, catalogue avec recherche, fiche produit
CP 4 — Capture du panier InnovShop mis à jour en AJAX
CP 5 — Schéma et liste des tables de la base InnovShop dans phpMyAdmin
CP 6 — Symfony Profiler : 2 requêtes sur le catalogue après la correction du problème N+1
CP 7 — Commande en 3 étapes, espace client, facture PDF, back-office EasyAdmin et résultat PHPUnit (17 tests)
CP 8 — Audit Lighthouse du site en production : 100 / 100 / 100 / 100
Les captures sont placées dans chaque exemple de pratique professionnelle, sous la rubrique concernée.

ANNEXES

(Si le RC le prévoit)

Liens à consulter :

- Site InnovShop en ligne : **https://hermann.alwaysdata.net/innovshop/**
- Code source InnovShop (GitHub) : **https://github.com/LettreH/innovshop**
- Tableau Kanban Trello InnovShop : **https://trello.com/b/CTkdf2jz/innovshop**
- Site du cabinet dentaire en ligne : **https://hermann.alwaysdata.net**
- Code source du cabinet dentaire (GitHub) : **https://github.com/LettreH/cabinet-dentaire**